

My report

Section A: Overview

The goal of this assignment was to develop a command line application that generates and manipulates magic squares. The first part, represented by Q1a_23025758, required creating an algorithm to generate an $n \times n$ magic square for odd integers n . The other part was the creation of a game for users, where they can move around and try to rebuild their magic square represented by Q1b_23025758.

Section B: Solution Design

To tackle the assignment, I created two Java programs: Q1a_23025758 and Q1b_23025758, each with a specific task. I have not explicitly made any assumptions beyond those set out in the assignment instructions. According to the instructions, the user must enter an odd number to determine the size of the magic square, and I used this assumption when designing the programs. For Q1a_23025758, the aim was to make a magic square for an odd number given by the user. I've been following the step-by-step approach laid down in this assignment. To organise the square's numbers, I use a gridlike structure referred to as a two-dimensional array. Then, after some rules were followed, I filled the square with numbers. Finally, I showed the finished magic square to the user.

Moving on to Q1b_23025758, the goal was to create a game where users interact with a shuffled magic square. To store the magic square, I used the same grid structure as before. The game allows users to mix the square and try to fix it by moving the numbers around. Users can make moves such as moving the number upwards, downwards, backwards or forwards. The program will keep track of your actions, telling the user when they are complete. I used a gridlike structure called a two-dimensional array, as suggested in the assignment, in both programs. This made it easier to organise and manipulate the magic squares. Overall, the solutions focused on simplicity and user-friendliness, in line with the assignment instructions.

Section C: Completed solution

The completed solution does two things which are creating a magic square for odd-sized matrices and giving a game where the user can shuffle and try to repute the magic square

For Q1a_23025758, I used the provided algorithm and a grid-like structure to arrange the numbers and for the game in q1b_23025758, the users can shuffle the magi square and then put it back together by stating the position and direction such as R, L, U, D. however, to improve I believe I could make the

code easier to understand by others by making a habit of writing comments and such. I could also make the shuffling algorithm more efficient to make sure the numbers in the square are mixed better.

Section D: Software testing

I tested my game on my laptop by trying to input a couple of odd numbers, inputting a specific direction and so on. I also tested providing invalid input to ensure that there is proper error handling, while I did not conduct formal unit tests, they could be added in the future to validate the program's functionality more. In conclusion, I

believe the implemented solution fulfils the assignment requirements by providing a command line magic square. In addition, I believe further improvements such as code readability could be added in the future.

```
sarahalrefaie@Sarahs-Laptop ~ % /usr/bin/env /opt/homebrew/Cellar/openjdk/
r/folders/2b/zh36r5td0hgfvz6rv0_0fdmxw0000gn/T/vscodews_c3b6f/jdt_ws/jdt.
Enter an odd number: 5
Magic Square of size 5 is created.
Magic Square after shuffling:
17 3 18 2 24
15 5 7 11 4
1 6 13 22 14
20 19 12 21 25
23 16 8 10 9

Choose to continue or to exit.
1. Enter position (i, j) in 0-index and a direction (U, R, L, D)
2. Exit
Choice: 1
Enter position (i, j) and direction:
4 4
R
After swapping:
17 3 18 2 24
15 5 7 11 4
1 6 13 22 14
20 19 12 21 25
9 16 8 10 23

Choose to continue or to exit.
1. Enter position (i, j) in 0-index and a direction (U, R, L, D)
2. Exit
Choice: 2

Total Moves = 1
Thanks for playing!
sarahalrefaie@Sarahs-Laptop ~ %
```